

T01 — Course Introduction & Metodologia del Corso

Obiettivi Formativi, Modalità d'Esame al Calcolatore, Toolchain e Syllabus

Docente: Prof. Michele Pasqua — **Appunti Revisione:** Wally

A.A. 2025/2026 – Università degli Studi di Verona

Benvenuto in questa revisione completa e metodica di **Programmazione II** (Prof. Michele Pasqua, UniVR 2025/2026). Seguiremo fedelmente le regole stabilite: **nessuna omissione**, analisi minuziosa di ogni concetto teorico, confronto riga per riga del codice, evidenziazione delle finezze d'esame e stop interattivo a ogni blocco.

0.0.1 1. Teoria: Concetti Teorici dalle Slide ([T01 - Course Introduction.pdf](#))

La slide introduttiva definisce il perimetro accademico, gli obiettivi formativi e le modalità d'esame:

1. Docente e Background Scientifico:

- **Prof. Michele Pasqua** (Dipartimento di Informatica, Cà Vignal 2, 2° piano, Stanza 24 — michele.pasqua@univr.it)
- Aree di ricerca: *Program verification and validation, Programming language design and implementation, Code protection and software security.*
- Laboratori e progetti: co-fondatore del laboratorio **UNIVERSE** (*University of Verona Software Engineering Lab*, diretto dal Prof. Mariano Ceccato) e co-fondatore dello spin-off universitario **KonTrust Srl** (tracciamento IoT su blockchain e infrastrutture anti-manomissione); referente per Verona del programma nazionale di formazione e gare su cybersecurity (*CyberChallenge.IT*).

2. Organizzazione della Didattica:

- **Modulo di Teoria (40+ ore):** Lunedì (16:30) e Martedì (13:30) in Aula A (2 ore ciascuna).
- **Modulo di Laboratorio (12+ ore):** Martedì (10:30) in Aula Delta (2 ore).
- **Materiale didattico di riferimento:**
 - Paul J. Deitel & Harvey M. Deitel, *Java – How to Program*, Pearson (11/e – 2018).
 - Claudio De Sio Cesari, *Il nuovo Java – Guida alla programmazione moderna*, Hoepli (1/e – 2020).
 - Moodle UniVR: slide, esercizi di laboratorio e soluzioni, simulazioni.

3. Modalità d'Esame e Valutazione:

- **Formato:** Prova pratica al calcolatore in **Aula Delta** (ambiente Linux/IDE).
- **Contenuto:** Risoluzione di una serie di esercizi di programmazione in linguaggio Java (sviluppo classi, gerarchie a oggetti, implementazione interfacce, algoritmi su collezioni, gestione eccezioni, test con asserzioni/JUnit).
- **Voto finale:** Somma algebrica dei punti ottenuti nei singoli esercizi.

- È prevista una **simulazione d'esame** ufficiale a fine corso.

4. Syllabus del Corso (Mappa degli argomenti da T01 a T20):

- *Object-Orientation Principles* (T02)
 - *Building, Running, and Deployment* (T03)
 - *Types, Classes, and Objects* (T04)
 - *Java Syntax and Semantics* (T05)
 - *Java Classes and Objects* (T06)
 - *Libraries and String Class* (T07)
 - *Class Constructors and Encapsulation* (T08)
 - *Arrays and Enumerative Types* (T09)
 - *Java Variables and Call Stack* (T10)
 - *Class and Package Visibility* (T11)
 - *Inheritance and Polymorphism* (T12)
 - *Abstract Classes and Interfaces* (T13)
 - *Nested and Anonymous Classes* (T14)
 - *Generic Types* (T15)
 - *Collections* (T16)
 - *Error Handling with Exceptions* (T17)
 - *Functional Interfaces and Streams* (T18)
 - *Documentation and Unit Testing* (T19)
 - *Java IO* (T20)
-

0.0.2 2. Codice: Analisi del Codice della Lezione 1

- Nella cartella ufficiale delle lezioni [Codice-20260902](#):
 - **Non sono presenti file di codice per le Lezioni 01, 02 e 03.**
 - Il primo codice sorgente ufficiale del docente compare a partire dalla **Lezione 04** con [Example.java](#) (dedicato a tipi primitivi, reference, casting e operatori), mentre il progetto a oggetti incrementale vero e proprio parte dal blocco **Lezioni 05-06** con il progetto SimpleDate ([Date.java](#) e [MainDate.java](#)).
 - Nel tuo workspace [esercizi-wally-25-26](#):
 - La cartella [es01](#) è già predisposta come modulo Maven con la prima incarnazione di [Date](#) e [MainDate](#), sincronizzata con la struttura multi-modulo orchestrata dal [pom.xml](#) di root e dallo script [esercizi.sh](#).
-

0.0.3 3. Setup: Configurazione Operativa dell'Ambiente

Per garantire che tutti i passaggi da qui in avanti siano riproducibili su IntelliJ IDEA e da CLI: 1. Verifica che la JDK attiva sia Java 17 o superiore: `bash java -version mvn -v` 2. Il file di root [pom.xml](#) gestisce l'aggregazione di tutti i sottomoduli. Per ogni blocco successivo, quando andremo ad analizzare le classi introdotte a lezione, verificheremo che compilino ed eseguano pulitamente sia tramite: `bash ./esercizi.sh run <numero_modulo>` sia all'interno di IntelliJ IDEA.

Nota di Studio

La **Lezione 1 / T01** esaurisce qui la parte introduttiva/organizzativa. Il prossimo blocco è la **Lezione 2 / Slide T02: *Object-Orientation and UML***, dove entreremo nel vivo con: - Storia e tassonomia dei paradigmi: *Imperativo/Procedurale, Dichiarativo/Funzionale, Object-Oriented*. - I 4 pilastri dell'OOP: *Astrazione, Incapsulamento, Ereditarietà, Polimorfismo*. - Concetto di *Classe* vs *Oggetto* (stato, comportamento, identità). - Notazione UML per classi e associazioni (*Visibility markers*, cardinalità, tipi di relazione).

Dammi conferma quando sei pronto per passare a T02 (procederemo analizzando il blocco concettuale un po' alla volta)!